

OGNL基础

目录

- OGNL概念
- OGNL基础知识
- OGNL表达式语法

一、OGNL概念

OGNL是什么？

OGNL (Object-Graph Navigation Language)，是一种功能强大的表达式语言，通过具有一定语法特性的语法，可以操作Java对象的属性、方法等。

为什么使用OGNL

Java是面向对象的语言，是强类型语言，但它又是可以开发Web程序，那么如何将HTTP中的数据无感的转为Java对象，使开发者无感知，从而更加专注与业务开发，这便是OGNL的作用

需要了解的前提

OGNL是一个脱离于Struts2库，并不是和Struts2绑定的，也就是说如果Struts2想使用OGNL，就必须满足OGNL的语法和环境需求

二、OGNL基础知识

首先看一个最简单的OGNL的使用案例，看代码是最直观的

```
1  # 命令执行调用calc
2  OgnlContext ognlContext = new OgnlContext();
3  Ognl.getValue("@java.lang.Runtime@getRuntime().exec('calc')",ognlContext,ognlContext.getRoot());
4  # 操作普通对象
5  User user = new User();
6  user.setUsername("user");
7  OgnlContext ognlContext = new OgnlContext();
8  ognlContext.setRoot(user);
9  String name = (String) Ognl.getValue("username",ognlContext,user);
10 System.out.println(name);
```

可以看到OGNL操作需要3个参数，也是3个要素，如果Struts2要使用OGNL，也必须得满足这3个要素

- expression：求值表达式-干什么
- rootobject：根对象-对谁干
- context：OGNL执行环境-在哪干

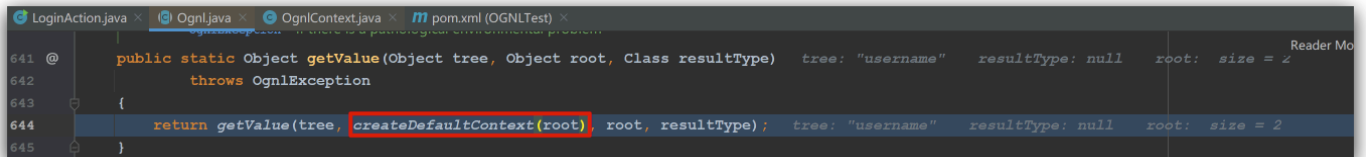
OgnlContext类实现了Map接口，所以它本质上是一个MAP

```
public class OgnlContext extends Object implements Map
```

主要接口如下

```
1  getValue(Object tree, Object root)
2  getValue(Object tree, Map context, Object root)
3  getValue(Object tree, Map context, Object root, Class resultType)
4  getValue(Object tree, Object root)
5  getValue(Object tree, Object root, Class resultType)
6  getValue(ExpressionAccessor expression, OgnlContext context, Object root)
7  getValue(ExpressionAccessor expression, OgnlContext context, Object root, Class
    resultType)
8  getValue(String expression, Map context, Object root)
9  getValue(String expression, Map context, Object root, Class resultType)
10 getValue(String expression, Object root)
11 getValue(String expression, Object root, Class resultType)
```

以上是OGNL所有getValue的重载方法，当参数中不传递OgnlContext时，便会自动创建一个空的OgnlContext对象，并设置root对象



```
641 @ public static Object getValue(Object tree, Object root, Class resultType) tree: "username" resultType: null root: size = 1 Reader Mo
642     throws OgnlException
643 {
644     return getValue(tree, createDefaultContext(root), root, resultType); tree: "username" resultType: null root: size = 2
645 }
```

那么下面的代码输出的内容应该是什么？

```
User user = new User();
user.setUsername("user");
OgnlContext ognlContext = new OgnlContext();
ognlContext.setRoot(user);

Object expression = Ognl.parseExpression("username");
String name = (String) Ognl.getValue(expression, ognlContext);
System.out.println(name);
```

答案是 null，因为我们没有传root对象，第二个参数是root对象，但我们却传了ognlContext，虽然没有爆异常，但却使root成为了Map，取不到数据。需要将代码换为

```
1  (String) Ognl.getValue(expression, ognlContext.getRoot());
2  或
3  (String) Ognl.getValue(expression, ognlContext, ognlContext.getRoot());
```

那么下面的代码输出的内容应该是什么？

```
User user = new User();
user.setUsername("user");

User user1 = new User();
user1.setUsername("user1");

OgnlContext ognlContext = new OgnlContext();
ognlContext.setRoot(user);
ognlContext.put("test", "testV");
ognlContext.put("user1", user1);

Object expression = Ognl.parseExpression("#user1['username']");
String name = (String) Ognl.getValue(expression, ognlContext, ognlContext.getRoot());
System.out.println(name);
```

答案是 `user1`，`#` 代表访问非root对象的属性，所以会在OGNL查找 `user1` 对象的 `username` 属性的值而 `#user1['username']` 便是OGNL表达式，接下来详细讲解下什么是OGNL表达式

三、OGNL表达式语法

在Struts2中经常遇到OGNL操作对象的符合，例如 `#`、`%`、`$`，这几个符合的作用分别是：

几个常见的操作符

#号符

1. 访问非root对象的属性，例如 `#user["username"]`
2. 加在this前面代表this指针
3. `#{}构造Map、List对象`

%号符

1. 在前端标签的属性值为字符串时，放在 `%{}` 内的字符串表示OGNL表达式，例如

```
<s:property value="%{#user['username']}" />
```

\$号符

1. 在配置文件中引用OGNL表达式，访问Action属性
2. 在国际化文件中引用OGNL表达式

@号符

1. 调用静态属性：@类的全称(含包名)@静态属性名
2. 调用静态方法：@类的全称(含包名)@静态方法名(参数列表)

投影与选择

这个名字起的有点不太好理解，其实就是数据的快速筛选

投影 (projection)：筛选出集合中每个元素的相同属性，组成新的集合，类似数据库查询某一列的所有值，例如 `group.userList.{username}`

选择 (selection)：选择就类似于查询数据库某一列符合条件的数据，组成新的集合，例如 `group.userList.{? #this.name != null}`

- **投影**：选出集合当中的相同属性组合成一个新的集合。语法为 `collection.{XXX}`，XXX 就是集合中每个元素的公共属性。
- **选择**：选择就是选择出集合当中符合条件的元素组合成新的集合。语法为 `collection.{Y XXX}`，其中 Y 是一个选择操作符，XXX 是选择用的逻辑表达式。

选择操作符有 3 种：

- `?`：选择满足条件的所有元素
- `^`：选择满足条件的第一个元素
- `$`：选择满足条件的最后一个元素

对象访问

对root对象访问

```
1 // 获取root对象的name的属性值
2 name
3 // 获取root对象中user对象的name属性的值
4 user.name
5 // 获取root对象中school对象的user对象的name属性的值
6 school.user.name
```

对OGNLContext上下文的访问

需要使用#符号访问，从而表示与访问Root对象的区别

```
1 // 获取OGNLContext中name的值
2 #name
3 // 获取OGNLContext中user对象的name属性的值
4 #user.name
```

对象的静态变量、方法的访问

在OGNL中，对于静态变量、静态方法的访问需要使用 `@[class]@[field / method]` 语法

```
1 // 获取User对象的的STATE属性
2 @com.example.User@STATE
3 // 调用User对象的getName方法
4 @com.example.User@getName()
```

在struts2-core的default.properties中默认禁止了对静态方法的调用，可以在struts.xml中修改为 `struts.ognl.allowStaticMethodAccess=true`

```
1 # Whether to allow static method access in OGNL expressions or not
2 struts.ognl.allowStaticMethodAccess=false
```

但是在struts2.3.20时，框架为了安全考虑，即便修改为true也不会起作用

对象的方法调用

在OGNL中，可以直接通过类似Java语法进行调用，例如

```
1 // 调用Root对象中的school对象中users的size()方法
2 school.users.size()
3 // 调用Root对象中的school中的containsUser的方法，并将上下文环境中名为requestUser的值作为参数传入
4 school.containsUser(#requestUser)
```

数组、List对象访问

```
1 // 访问下标为0的对象的name属性
2 school.users[0].name
```

创建对象

```
1 // 创建普通对象
2 #name=new java.lang.String('aa')
3 // 创建MAP
4 #{'name':'tom','age':18}
5 // 创建List
6 #{'tom','jerry','jack','rose'}
```

在OGNL表达式中自由的使用Java对象

下面的代码是使用OGNL定义了一个String变量，并将其打印出来，明白这点之后，便可以慢慢的理解所有的Struts2漏洞的利用代码 使用Java类一定要加package全路径 里面的点号其实没有什么实际意义，只是为了满足语法

```
1 Ognl.getValue("#name=new java.lang.String('aa')).
  (@java.lang.System@out.println(#name))",ognlContext,ognlContext.getRoot());
```

```
public void testOGNL1() throws OgnlException {
    OgnlContext ognlContext = new OgnlContext();
    Ognl.getValue("@java.lang.Runtime@getRuntime().exec('calc')",ognlContext,ognlContext.getRoot());
    Ognl.getValue( expression: "#name=new java.lang.String('aa')). (@java.lang.System@out.println(#name))",ognlContext,ognlContext.getRoot());
}
```

四、OGNL节点处理对象

SimpleNode是构成AST树的父类，每个子类都对应了不同语法的解析方式，在toString方法中可以看出该类解析的哪一种表达式，[官方文档点击这里](#)，下面对SimpleNode的所有子类进行详细解释：

1. ASTEval：用于处理()
格式
2. ExpressionNode：用于处理加减乘除、异或运算等基础运算
3. ASTList：用于处理OGNL的列表，例如 {12,3}，会返回一个List对象
4. ASTVarRef：用于处理变量的定义，例如 #person
5. ASTChain：用于处理链式调用，例如： #person.name 、 #person.say()
6. ASTStaticField：处理静态属性，例如 @org.example.StaticPerson@sex
7. ASTMap：处理Map，例如： #{ "name": "xiaoming", "school": "tsinghua" }
8. ASTSequence：处理逗号分隔的表达式，例如：
如： Ognl.getValue("name,#person1.say()",context,person)，表达式中的name和say都会调用
9. ASTSelectLast：从集合中拿到最后一个符合规则的元素

```
1  OgnlContext context = new OgnlContext();
2  List list = new ArrayList();
3  list.add(new Integer(1));
4  list.add(new String("a"));
5  list.add(new Integer(2));
6  list.add(new String("b"));
7  context.put("list", list);
8  System.out.println(Ognl.getValue("#list.{ $ #this instanceof
String}",context,"").toString());
9
10 # 运行结果为
11 [b]
```

10. ASTSelect：从集合中选中符合规则的元素，重新组合为一个新的集合

```
1  OgnlContext context = new OgnlContext();
2  List list = new ArrayList();
3  list.add(new Integer(1));
4  list.add(new String("a"));
5  list.add(new Integer(2));
6  list.add(new String("b"));
7  context.put("list", list);
8  System.out.println(Ognl.getValue("#list.{ ? #this instanceof
String}",context,"").toString());
9
10 // 运行结果为
11 [a, b]
```

11. ASTKeyValue: 也是处理Map, 用key->value的形式表示Map

12. ASTSelectFirst: 返回第一个符合规则的元素, 例如: `objects.{^ #this instanceof String }`

13. ASTNotIn: 某个值不在某个集合中; ASTIn: 某个值在某个集合中

```
1  OgnlContext context = new OgnlContext();
2  List list = new ArrayList();
3  list.add("a");
4  list.add("b");
5  context.put("list", list);
6  context.put("a", "a");
7  System.out.println(Ognl.getValue("#a not in #list", context, "").toString());
8
9  // 运行结果, 如果not in 改为in则为true
10 false
```

14. ASTMethod: 调用方法, 例如 `#person.say()`

15. ASTProject: OGNL的投影特性

```
1  OgnlContext context = new OgnlContext();
2  List list = new ArrayList();
3  list.add(new Person());
4  list.add(new Person());
5  list.add(new Person());
6  list.add(new Person());
7  context.put("list", list);
8  System.out.println(Ognl.getValue("#list.{name}", context, "").toString());
9
10 // 运行结果, 取出所有的name
11 [test, test, test, test]
```

16. ASTInstanceOf: 对象比较

```
1  OgnlContext context = new OgnlContext();
2  List list = new ArrayList();
3  list.add(new Person());
4  list.add(new Person());
5  context.put("list", list);
6  System.out.println(Ognl.getValue("#list.{? #this instanceof
   org.example.Person}", context, "").toString());
7
8  // 运行结果
9  [org.example.Person@30dae81, org.example.Person@1b2c6ec2]
```

17. ASTStaticMethod: 处理静态方法, 例如: `@org.example.StaticPerson@say()`

18. ASTCtor: 处理数组, 例如: `#p=(new java.lang.ProcessBuilder(new java.lang.String[]
{"calc"})).start()`

19. ASTConst: 普通常量, 例如数字、字符串

20. ASTProperty: 获取对象属性

```
1  OgnlContext context = new OgnlContext();
2  String[] str = new String[]{"1","2","3"};
3  context.put("str", str);
4  System.out.println(Ognl.getValue("#str['length']",context,"").toString());
5
6  // 运行结果
7  3
```

21. ASTAssign: 赋值运算, 例如 `#str=1`